

Enterprise CRM Integrations Architecture

Overview

The CRM Integrations module provides comprehensive, real-time bidirectional synchronization with enterprise CRM systems, focusing on Salesforce and HubSpot. This architecture ensures seamless data flow, conflict resolution, and maintains data integrity across all connected systems while supporting high-volume enterprise operations.

Integration Architecture

Core Integration Framework

```
class CRMIntegrationFramework:
    def __init__(self):
        self.salesforce_connector = SalesforceConnector()
        self.hubspot_connector = HubSpotConnector()
        self.sync_orchestrator = SyncOrchestrator()
        self.conflict_resolver = ConflictResolver()
        self.data_mapper = DataMapper()
        self.audit_logger = AuditLogger()

    async def initialize_integrations(self):
        # Initialize all CRM connections
        await self.salesforce_connector.initialize()
        await self.hubspot_connector.initialize()

        # Start sync orchestration
        await self.sync_orchestrator.start()

        # Begin audit logging
        await self.audit_logger.start_logging()
```

1. Salesforce Integration

Advanced Salesforce Connector

Multi-API Integration:

```

class SalesforceConnector:
    def __init__(self):
        self.rest_client = SalesforceRESTClient()
        self.bulk_client = SalesforceBulkClient()
        self.streaming_client = SalesforceStreamingClient()
        self.metadata_client = SalesforceMetadataClient()
        self.composite_client = SalesforceCompositeClient()

    async def initialize(self):
        # OAuth 2.0 authentication with refresh token handling
        auth_config = {
            "client_id": config.SALESFORCE_CLIENT_ID,
            "client_secret": config.SALESFORCE_CLIENT_SECRET,
            "username": config.SALESFORCE_USERNAME,
            "password": config.SALESFORCE_PASSWORD,
            "security_token": config.SALESFORCE_SECURITY_TOKEN,
            "domain": config.SALESFORCE_DOMAIN
        }

        # Initialize all API clients
        await self.rest_client.authenticate(auth_config)
        await self.bulk_client.authenticate(auth_config)
        await self.streaming_client.authenticate(auth_config)
        await self.metadata_client.authenticate(auth_config)

        # Set up streaming API subscriptions
        await self.setup_streaming_subscriptions()

    async def setup_streaming_subscriptions(self):
        # Subscribe to real-time changes
        subscriptions = [
            "/data/AccountChangeEvent",
            "/data/ContactChangeEvent",
            "/data/OpportunityChangeEvent",
            "/data/LeadChangeEvent",
            "/data/TaskChangeEvent",
            "/data/EventChangeEvent"
        ]

        for subscription in subscriptions:
            await self.streaming_client.subscribe(
                subscription,
                callback=self.handle_streaming_event
            )

```

Real-Time Data Synchronization:

```

class SalesforceRealTimeSync:
    def __init__(self):
        self.change_data_capture = ChangeDataCapture()
        self.platform_events = PlatformEvents()
        self.sync_queue = AsyncQueue()

    async def handle_streaming_event(self, event):
        # Process Salesforce streaming events
        event_type = event.get("event", {}).get("type")

        if event_type == "created":
            await self.handle_record_creation(event)
        elif event_type == "updated":
            await self.handle_record_update(event)
        elif event_type == "deleted":
            await self.handle_record_deletion(event)

    async def handle_record_update(self, event):
        # Extract change information
        record_id = event["data"]["payload"]["Id"]
        object_type = event["data"]["payload"]["ChangeEventHeader"]["entityName"]
        changed_fields = event["data"]["payload"]["ChangeEventHeader"]["changedFields"]

        # Create sync task
        sync_task = SyncTask(
            operation="update",
            source_system="salesforce",
            target_systems=["hubspot", "internal_db"],
            record_id=record_id,
            object_type=object_type,
            changed_fields=changed_fields,
            timestamp=datetime.utcnow(),
            priority="high"
        )

        await self.sync_queue.put(sync_task)

```

Bulk Operations Handler:

```

class SalesforceBulkOperations:
    def __init__(self):
        self.bulk_client = SalesforceBulkClient()
        self.batch_processor = BatchProcessor()

    async def bulk_upsert(self, records, object_type):
        # Prepare bulk operation
        job = await self.bulk_client.create_job(
            operation="upsert",
            object_type=object_type,
            external_id_field="External_Id__c"
        )

        # Process in batches
        batch_size = 10000
        batches = self.batch_processor.create_batches(records, batch_size)

        batch_results = []
        for batch in batches:
            batch_result = await self.bulk_client.add_batch(
                job_id=job.id,
                data=batch
            )
            batch_results.append(batch_result)

        # Close job and wait for completion
        await self.bulk_client.close_job(job.id)

        # Monitor job progress
        job_status = await self.monitor_job_completion(job.id)

        return BulkOperationResult(
            job_id=job.id,
            batch_results=batch_results,
            job_status=job_status,
            records_processed=len(records)
        )

```

Custom Object Management

Dynamic Object Handling:

```

class SalesforceCustomObjectManager:
    def __init__(self):
        self.metadata_client = SalesforceMetadataClient()
        self.object_cache = ObjectMetadataCache()

    async def discover_custom_objects(self):
        # Retrieve all custom objects
        describe_result = await self.metadata_client.describe_global()

        custom_objects = []
        for object in describe_result.objects:
            if object.custom and object.createable:
                object_metadata = await self.metadata_client.describe_object(
                    object.name
                )
                custom_objects.append(object_metadata)

        # Cache metadata for performance
        await self.object_cache.cache_metadata(custom_objects)

        return custom_objects

    async def create_ai_insight_objects(self):
        # Create custom objects for AI-generated insights
        ai_insight_metadata = {
            "fullName": "AI_Sales_Insight__c",
            "label": "AI Sales Insight",
            "pluralLabel": "AI Sales Insights",
            "nameField": {
                "type": "AutoNumber",
                "label": "Insight Number"
            },
            "deploymentStatus": "Deployed",
            "sharingModel": "ReadWrite",
            "fields": [
                {
                    "fullName": "Account__c",
                    "type": "Lookup",
                    "relationshipName": "Account",
                    "referenceTo": ["Account"],
                    "label": "Account"
                },
                {
                    "fullName": "Insight_Type__c",
                    "type": "Picklist",
                    "label": "Insight Type",
                    "valueSet": {
                        "valueSetDefinition": {
                            "value": [
                                {"fullName": "Next Best Action"},
                                {"fullName": "Risk Assessment"},
                                {"fullName": "Opportunity Prediction"},
                                {"fullName": "Conversation Summary"}
                            ]
                        }
                    }
                }
            ],
            {
                "fullName": "Insight_Content__c",
                "type": "LongTextArea",
                "length": 32768,
                "label": "Insight Content"
            }
        }

```

```
        },  
        {  
            "fullName": "Confidence_Score__c",  
            "type": "Number",  
            "precision": 5,  
            "scale": 2,  
            "label": "Confidence Score"  
        }  
    ]  
}  
  
return await self.metadata_client.create_custom_object(ai_insight_metadata)
```

2. HubSpot Integration

HubSpot Private App Architecture

Comprehensive HubSpot Connector:

```

class HubSpotConnector:
    def __init__(self):
        self.private_app_client = HubSpotPrivateAppClient()
        self.webhook_handler = HubSpotWebhookHandler()
        self.batch_client = HubSpotBatchClient()
        self.timeline_client = HubSpotTimelineClient()

    async def initialize(self):
        # Initialize private app with comprehensive scopes
        scopes = [
            "contacts.read", "contacts.write",
            "companies.read", "companies.write",
            "deals.read", "deals.write",
            "tickets.read", "tickets.write",
            "timeline.read", "timeline.write",
            "crm.objects.custom.read", "crm.objects.custom.write",
            "crm.schemas.custom.read", "crm.schemas.custom.write"
        ]

        await self.private_app_client.initialize(
            access_token=config.HUBSPOT_ACCESS_TOKEN,
            scopes=scopes
        )

        # Set up webhook subscriptions
        await self.setup_webhooks()

    async def setup_webhooks(self):
        webhook_subscriptions = [
            {
                "eventType": "contact.creation",
                "propertyName": "*"
            },
            {
                "eventType": "contact.propertyChange",
                "propertyName": "*"
            },
            {
                "eventType": "company.creation",
                "propertyName": "*"
            },
            {
                "eventType": "deal.creation",
                "propertyName": "*"
            },
            {
                "eventType": "deal.propertyChange",
                "propertyName": "dealstage"
            }
        ]

        for subscription in webhook_subscriptions:
            await self.webhook_handler.create_subscription(
                subscription,
                callback_url=f"{config.BASE_URL}/webhooks/hubspot"
            )

```

Real-Time Webhook Processing:

```

class HubSpotWebhookHandler:
    def __init__(self):
        self.signature_validator = WebhookSignatureValidator()
        self.event_processor = EventProcessor()
        self.sync_queue = AsyncQueue()

    async def handle_webhook(self, request):
        # Validate webhook signature
        if not await self.signature_validator.validate(request):
            raise WebhookValidationError("Invalid webhook signature")

        # Parse webhook payload
        events = request.json()

        for event in events:
            await self.process_webhook_event(event)

    async def process_webhook_event(self, event):
        event_type = event.get("subscriptionType")
        object_id = event.get("objectId")

        if event_type == "contact.propertyChange":
            await self.handle_contact_property_change(event)
        elif event_type == "deal.creation":
            await self.handle_deal_creation(event)
        elif event_type == "company.creation":
            await self.handle_company_creation(event)

    async def handle_contact_property_change(self, event):
        # Extract change details
        contact_id = event["objectId"]
        changed_properties = event.get("propertyName", [])

        # Fetch full contact data
        contact_data = await self.private_app_client.contacts.get_by_id(
            contact_id,
            properties=changed_properties
        )

        # Create sync task
        sync_task = SyncTask(
            operation="update",
            source_system="hubspot",
            target_systems=["salesforce", "internal_db"],
            record_id=contact_id,
            object_type="contact",
            data=contact_data,
            timestamp=datetime.utcnow()
        )

        await self.sync_queue.put(sync_task)

```

Custom Properties and Objects

Dynamic Property Management:

```

class HubSpotCustomPropertyManager:
    def __init__(self):
        self.properties_client = HubSpotPropertiesClient()
        self.schemas_client = HubSpotSchemasClient()

    async def create_ai_properties(self):
        # Create custom properties for AI insights
        ai_properties = [
            {
                "name": "ai_lead_score",
                "label": "AI Lead Score",
                "type": "number",
                "fieldType": "number",
                "groupName": "ai_insights",
                "description": "AI-generated lead score based on conversation analysis",
                "options": []
            },
            {
                "name": "ai_next_best_action",
                "label": "AI Next Best Action",
                "type": "string",
                "fieldType": "textarea",
                "groupName": "ai_insights",
                "description": "AI-recommended next best action for this contact"
            },
            {
                "name": "ai_sentiment_score",
                "label": "AI Sentiment Score",
                "type": "number",
                "fieldType": "number",
                "groupName": "ai_insights",
                "description": "Overall sentiment score from conversation analysis"
            }
        ]

        created_properties = []
        for property_config in ai_properties:
            created_property = await self.properties_client.create(
                object_type="contacts",
                property_config=property_config
            )
            created_properties.append(created_property)

        return created_properties

```

3. Real-Time Bidirectional Synchronization

Sync Orchestration Engine

Advanced Sync Orchestrator:

```

class SyncOrchestrator:
    def __init__(self):
        self.sync_queue = PriorityQueue()
        self.conflict_resolver = ConflictResolver()
        self.data_mapper = DataMapper()
        self.sync_workers = []
        self.sync_monitor = SyncMonitor()

    async def start(self):
        # Start sync worker pool
        for i in range(config.SYNC_WORKER_COUNT):
            worker = SyncWorker(
                worker_id=i,
                sync_queue=self.sync_queue,
                conflict_resolver=self.conflict_resolver,
                data_mapper=self.data_mapper
            )
            self.sync_workers.append(worker)
            asyncio.create_task(worker.start())

        # Start sync monitoring
        asyncio.create_task(self.sync_monitor.start())

    async def queue_sync_task(self, sync_task):
        # Calculate priority based on business rules
        priority = self.calculate_sync_priority(sync_task)
        sync_task.priority = priority

        # Add to queue
        await self.sync_queue.put((priority, sync_task))

        # Log sync task
        await self.sync_monitor.log_sync_task(sync_task)

    def calculate_sync_priority(self, sync_task):
        # Priority calculation based on multiple factors
        base_priority = 100

        # Object type priority
        object_priorities = {
            "opportunity": 50,
            "lead": 40,
            "contact": 30,
            "account": 20,
            "task": 10
        }

        priority = base_priority + object_priorities.get(
            sync_task.object_type.lower(), 0
        )

        # Urgency modifiers
        if sync_task.operation == "delete":
            priority += 30
        elif sync_task.operation == "create":
            priority += 20
        elif sync_task.operation == "update":
            priority += 10

        # Time-based priority boost
        age_minutes = (datetime.utcnow() - sync_task.timestamp).total_seconds() / 60
        if age_minutes > 60: # Older than 1 hour

```

```
priority += int(age_minutes / 10)

return priority
```

Sync Worker Implementation:

```

class SyncWorker:
    def __init__(self, worker_id, sync_queue, conflict_resolver, data_mapper):
        self.worker_id = worker_id
        self.sync_queue = sync_queue
        self.conflict_resolver = conflict_resolver
        self.data_mapper = data_mapper
        self.is_running = False

    async def start(self):
        self.is_running = True
        logger.info(f"Sync worker {self.worker_id} started")

        while self.is_running:
            try:
                # Get next sync task
                priority, sync_task = await self.sync_queue.get()

                # Process sync task
                await self.process_sync_task(sync_task)

                # Mark task as done
                self.sync_queue.task_done()

            except Exception as e:
                logger.error(f"Sync worker {self.worker_id} error: {e}")
                await asyncio.sleep(1)

    async def process_sync_task(self, sync_task):
        try:
            # Map data between systems
            mapped_data = await self.data_mapper.map_data(
                source_system=sync_task.source_system,
                target_systems=sync_task.target_systems,
                data=sync_task.data,
                object_type=sync_task.object_type
            )

            # Check for conflicts
            conflicts = await self.conflict_resolver.detect_conflicts(
                sync_task, mapped_data
            )

            if conflicts:
                # Resolve conflicts
                resolved_data = await self.conflict_resolver.resolve_conflicts(
                    conflicts, mapped_data
                )
            else:
                resolved_data = mapped_data

            # Execute sync operations
            sync_results = []
            for target_system in sync_task.target_systems:
                result = await self.execute_sync_operation(
                    target_system=target_system,
                    operation=sync_task.operation,
                    data=resolved_data[target_system],
                    object_type=sync_task.object_type
                )
                sync_results.append(result)

            # Log successful sync

```

```
        await self.log_sync_success(sync_task, sync_results)

    except Exception as e:
        # Log sync failure
        await self.log_sync_failure(sync_task, str(e))

    # Retry logic
    if sync_task.retry_count < config.MAX_SYNC_RETRIES:
        sync_task.retry_count += 1
        sync_task.next_retry = datetime.utcnow() + timedelta(
            minutes=2 ** sync_task.retry_count
        )
        await self.sync_queue.put((sync_task.priority - 10, sync_task))
```

Conflict Resolution System

Advanced Conflict Resolver:

```

class ConflictResolver:
    def __init__(self):
        self.conflict_rules = ConflictResolutionRules()
        self.manual_review_queue = ManualReviewQueue()

    async def detect_conflicts(self, sync_task, mapped_data):
        conflicts = []

        # Timestamp-based conflict detection
        for target_system, data in mapped_data.items():
            existing_record = await self.get_existing_record(
                target_system, sync_task.record_id, sync_task.object_type
            )

            if existing_record:
                # Check for concurrent modifications
                if (existing_record.last_modified > sync_task.timestamp and
                    existing_record.last_modified_by != sync_task.source_system):

                    conflict = DataConflict(
                        conflict_type="concurrent_modification",
                        source_system=sync_task.source_system,
                        target_system=target_system,
                        source_data=sync_task.data,
                        target_data=existing_record.data,
                        conflict_fields=self.identify_conflicting_fields(
                            sync_task.data, existing_record.data
                        )
                    )
                    conflicts.append(conflict)

        return conflicts

    async def resolve_conflicts(self, conflicts, mapped_data):
        resolved_data = mapped_data.copy()

        for conflict in conflicts:
            resolution_strategy = await self.conflict_rules.get_resolution_strategy(
                conflict
            )

            if resolution_strategy == "source_wins":
                # Keep source data
                continue
            elif resolution_strategy == "target_wins":
                # Use target data
                resolved_data[conflict.target_system] = conflict.target_data
            elif resolution_strategy == "merge":
                # Merge data intelligently
                merged_data = await self.merge_data(
                    conflict.source_data,
                    conflict.target_data,
                    conflict.conflict_fields
                )
                resolved_data[conflict.target_system] = merged_data
            elif resolution_strategy == "manual_review":
                # Queue for manual review
                await self.manual_review_queue.add_conflict(conflict)
                # Skip sync for now
                del resolved_data[conflict.target_system]

        return resolved_data

```

4. Data Mapping and Transformation

Intelligent Data Mapper

Schema-Aware Data Mapping:

```
class DataMapper:
    def __init__(self):
        self.mapping_rules = MappingRules()
        self.schema_registry = SchemaRegistry()
        self.transformation_engine = TransformationEngine()

    async def map_data(self, source_system, target_systems, data, object_type):
        mapped_data = {}

        # Get source schema
        source_schema = await self.schema_registry.get_schema(
            source_system, object_type
        )

        for target_system in target_systems:
            # Get target schema
            target_schema = await self.schema_registry.get_schema(
                target_system, object_type
            )

            # Get mapping rules
            mapping_rule = await self.mapping_rules.get_mapping(
                source_system, target_system, object_type
            )

            # Transform data
            transformed_data = await self.transformation_engine.transform(
                source_data=data,
                source_schema=source_schema,
                target_schema=target_schema,
                mapping_rule=mapping_rule
            )

            mapped_data[target_system] = transformed_data

        return mapped_data
```

Field-Level Mapping Rules:

```

class MappingRules:
    def __init__(self):
        self.rules_store = MappingRulesStore()

    async def get_mapping(self, source_system, target_system, object_type):
        # Load mapping configuration
        mapping_key = f"{source_system}_{target_system}_{object_type}"
        mapping_config = await self.rules_store.get_mapping(mapping_key)

        if not mapping_config:
            # Generate default mapping
            mapping_config = await self.generate_default_mapping(
                source_system, target_system, object_type
            )

        return mapping_config

    async def generate_default_mapping(self, source_system, target_system, ob-
ject_type):
        # Intelligent field mapping based on field names and types
        source_fields = await self.get_object_fields(source_system, object_type)
        target_fields = await self.get_object_fields(target_system, object_type)

        field_mappings = {}

        for source_field in source_fields:
            # Direct name match
            if source_field.name in [tf.name for tf in target_fields]:
                field_mappings[source_field.name] = source_field.name
                continue

            # Fuzzy matching
            best_match = self.find_best_field_match(source_field, target_fields)
            if best_match and best_match.confidence > 0.8:
                field_mappings[source_field.name] = best_match.field_name

        return MappingConfiguration(
            source_system=source_system,
            target_system=target_system,
            object_type=object_type,
            field_mappings=field_mappings,
            transformation_rules=self.generate_transformation_rules(field_mappings)
        )

```

5. API Quota Management and Optimization

Intelligent API Management

API Quota Manager:

```

class APIQuotaManager:
    def __init__(self):
        self.quota_tracker = QuotaTracker()
        self.rate_limiter = RateLimiter()
        self.request_optimizer = RequestOptimizer()

    async def execute_api_request(self, system, request_type, request_data):
        # Check quota availability
        quota_status = await self.quota_tracker.check_quota(system, request_type)

        if not quota_status.has_quota:
            # Queue request for later execution
            await self.queue_request_for_later(system, request_type, request_data)
            raise QuotaExceededException(f"No quota available for {system}")

        # Apply rate limiting
        await self.rate_limiter.wait_if_needed(system, request_type)

        # Optimize request
        optimized_request = await self.request_optimizer.optimize(
            system, request_type, request_data
        )

        # Execute request
        try:
            response = await self.execute_request(system, optimized_request)

            # Update quota tracking
            await self.quota_tracker.record_usage(
                system, request_type, response.quota_used
            )

            return response

        except APIException as e:
            # Handle API errors
            await self.handle_api_error(system, request_type, e)
            raise

```

Request Optimization:

```

class RequestOptimizer:
    def __init__(self):
        self.batch_optimizer = BatchOptimizer()
        self.field_optimizer = FieldOptimizer()

    async def optimize(self, system, request_type, request_data):
        optimized_data = request_data.copy()

        # Batch optimization
        if request_type in ["create", "update", "upsert"]:
            optimized_data = await self.batch_optimizer.optimize_batch(
                system, optimized_data
            )

        # Field optimization
        if request_type in ["read", "query"]:
            optimized_data = await self.field_optimizer.optimize_fields(
                system, optimized_data
            )

        return optimized_data

    async def optimize_batch(self, system, request_data):
        # Combine multiple requests into batch operations
        if system == "salesforce" and len(request_data) > 1:
            # Use Salesforce Composite API
            return self.create_composite_request(request_data)
        elif system == "hubspot" and len(request_data) > 1:
            # Use HubSpot Batch API
            return self.create_batch_request(request_data)

        return request_data

```

6. Error Handling and Recovery

Comprehensive Error Management

Error Handler:

```

class CRMErrorHandler:
    def __init__(self):
        self.retry_manager = RetryManager()
        self.circuit_breaker = CircuitBreaker()
        self.error_classifier = ErrorClassifier()

    async def handle_error(self, error, context):
        # Classify error type
        error_type = await self.error_classifier.classify(error)

        if error_type == "transient":
            # Retry with exponential backoff
            return await self.retry_manager.retry_with_backoff(
                context.operation, context.data
            )
        elif error_type == "rate_limit":
            # Wait and retry
            await self.handle_rate_limit_error(error, context)
        elif error_type == "authentication":
            # Refresh authentication and retry
            await self.refresh_authentication(context.system)
            return await self.retry_manager.retry_once(
                context.operation, context.data
            )
        elif error_type == "permanent":
            # Log error and move to dead letter queue
            await self.handle_permanent_error(error, context)
        else:
            # Unknown error - circuit breaker
            await self.circuit_breaker.record_failure(context.system)
            raise error

```

This comprehensive CRM integration architecture ensures reliable, scalable, and intelligent synchronization between enterprise CRM systems while maintaining data integrity and providing robust error handling and recovery mechanisms.